



[Home](#)

[Content](#)

[Contact](#)

[Others](#)

PROJETO GUIADO – PARTE 2

Construção, comparação e
escolha do melhor modelo

Grupo de Estudo de Machine Learning – Panda UFSCar 2026

Objetivos da semana

- Construir **múltiplos modelos** de Machine Learning
- Comparar o desempenho de cada modelo
- Escolher o **melhor modelo** para o problema
- **Documentar** todo o processo e os resultados
- Preparar o projeto final para o **Workshop Panda**

Essa é a semana de **finalizar** o projeto!



O que fizemos na semana passada

Etapa	O que fizemos
1. Dataset	Cada um escolheu seu próprio dataset
2. EDA	Análise exploratória, visualizações, insights
3. Pré-processamento	Tratar nulos, escalonar, codificar categorias
4. Baseline	Modelo simples para servir de referência
5. Documentação	Notebook com o passo a passo

Agora: Vamos construir modelos melhores e escolher o melhor!

O que vamos fazer hoje

O roteiro de hoje

Etapa	O que vamos fazer
1	Escolher vários modelos para testar
2	Treinar cada modelo com os dados de treino
3	Avaliar cada modelo com os dados de teste
4	Comparar os resultados (métricas)
5	Escolher o melhor modelo para o problema
6	Documentar tudo no notebook

Vamos ver cada etapa em detalhes!

Modelos para testar

Classificação:

Modelo	Característica
Regressão Logística	Simples, rápido, interpretável
Árvore de Decisão	Interpretável, não linear
Random Forest	Melhor que árvore simples, robusto
KNN	Não paramétrico, baseado em distância
XGBoost	Muito poderoso, mas mais complexo

Regressão:

Modelo	Característica
Regressão Linear	Simples, rápido, interpretável
Árvore de Decisão	Não linear, interpretável
Random Forest	Melhor que árvore simples
XGBoost	Muito poderoso, mas mais complexo

Escolham 3-5 modelos para testar. Não precisa ser todos!

Como comparar modelos?

Classificação:

Métrica	O que mede	Como comparar
Acurácia	Percentual total de acertos	Quanto maior, melhor
Precisão	Confiança no SIM	Quanto maior, melhor
Recall	Capacidade de detectar SIMs	Quanto maior, melhor
F1-Score	Média harmônica entre Precisão e Recall	Quanto maior, melhor

Regressão:

Métrica	O que mede	Como comparar
R²	Proporção da variância explicada	Quanto mais perto de 1, melhor
MSE	Erro quadrático médio	Quanto menor, melhor
MAE	Erro absoluto médio	Quanto menor, melhor
RMSE	Raiz do erro quadrático médio	Quanto menor, melhor

A métrica mais importante depende do problema!

Exemplo de código – Classificação

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Lista de modelos
modelos = {
    'Regressão Logística': LogisticRegression(),
    'Árvore de Decisão': DecisionTreeClassifier(),
    'Random Forest': RandomForestClassifier()
}

# Treinar e avaliar cada modelo
resultados = {}
for nome, modelo in modelos.items():
    modelo.fit(X_treino, y_treino)
    pred = modelo.predict(X_teste)

    resultados[nome] = {
        'Acurácia': accuracy_score(y_teste, pred),
        'Precisão': precision_score(y_teste, pred, average='weighted'),
        'Recall': recall_score(y_teste, pred, average='weighted'),
        'F1': f1_score(y_teste, pred, average='weighted')
    }

# Ver resultados
df_resultados = pd.DataFrame(resultados).T
print(df_resultados)
```



Exemplo de código – Regressão

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Lista de modelos
modelos = {
    'Regressão Linear': LinearRegression(),
    'Árvore de Decisão': DecisionTreeRegressor(),
    'Random Forest': RandomForestRegressor()
}

# Treinar e avaliar cada modelo
resultados = {}
for nome, modelo in modelos.items():
    modelo.fit(X_treino, y_treino)
    pred = modelo.predict(X_teste)

    resultados[nome] = {
        'R²': r2_score(y_teste, pred),
        'MSE': mean_squared_error(y_teste, pred),
        'MAE': mean_absolute_error(y_teste, pred)
    }

# Ver resultados
df_resultados = pd.DataFrame(resultados).T
print(df_resultados)
```



Qual modelo escolher?

Perguntas para ajudar na decisão:

1. Qual modelo teve a melhor métrica principal?

- Para classificação: acurácia, precisão, recall ou F1?
- Para regressão: R^2 , MSE ou MAE?

2. Os resultados são consistentes?

- O modelo foi avaliado com Cross-Validation?
- As métricas são parecidas entre treino e teste?

3. O modelo é interpretável?

- Se for para apresentar para pessoas não técnicas, isso importa
- Regressão Logística e Árvores são mais fáceis de explicar

4. O modelo é rápido o suficiente?

- Para um projeto pequeno, isso não é tão crítico

Geralmente: O modelo que tem a melhor métrica principal e é consistente é o escolhido.



Como melhorar o modelo escolhido

Depois de escolher o modelo, você pode:

Estratégia	Como fazer
Ajustar hiperparâmetros	Usar GridSearch nos hiperparâmetros do modelo escolhido
Adicionar mais features	Criar novas variáveis (feature engineering)
Remover features irrelevantes	Selecionar apenas as mais importantes
Balancear os dados	Se houver desbalanceamento (ex: 90% de uma classe)

Recomendação: Aplicar GridSearch no modelo escolhido para ter o melhor desempenho possível.

GridSearch no modelo escolhido

```
from sklearn.model_selection import GridSearchCV

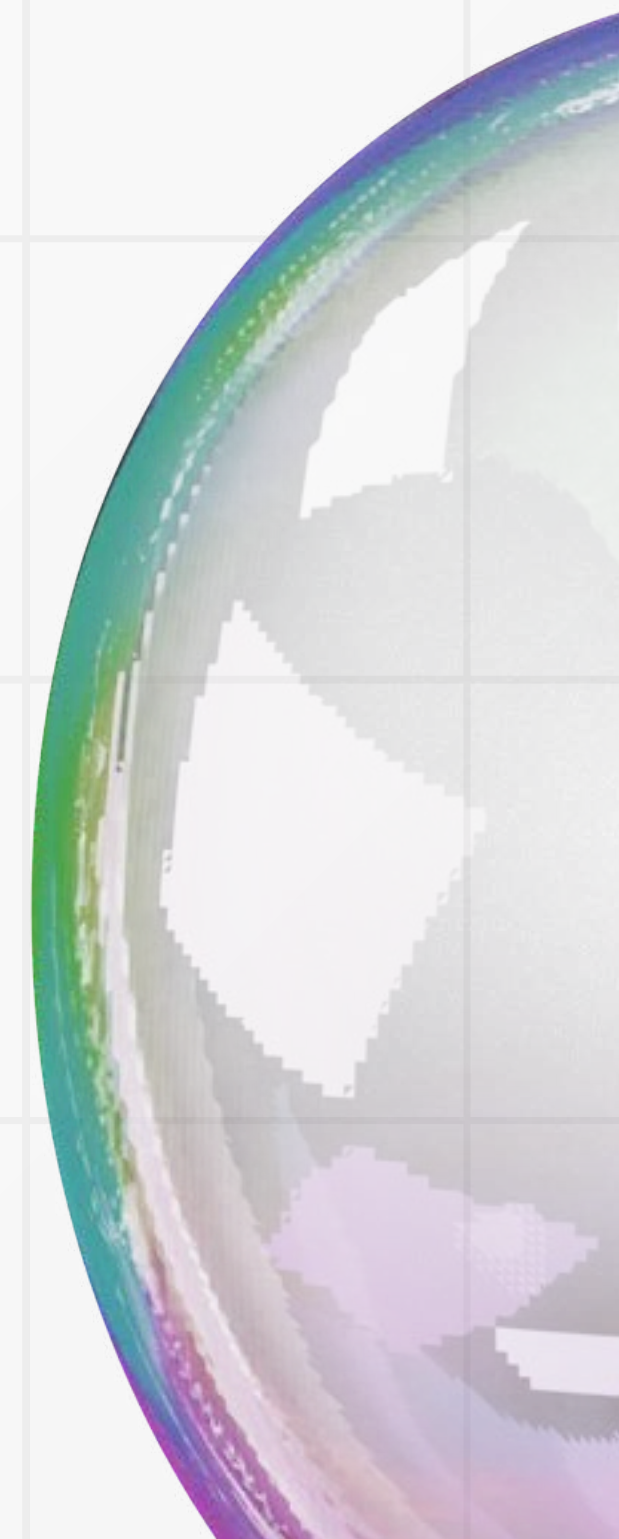
# Definir o modelo
modelo = RandomForestClassifier()

# Definir hiperparâmetros para testar
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10]
}

# GridSearch com Cross-Validation
grid = GridSearchCV(modelo, param_grid, cv=5)
grid.fit(X_treino, y_treino)

print(f'Melhores parâmetros: {grid.best_params_}')
print(f'Melhor acurácia média: {grid.best_score_:.2f}')

# Avaliar no teste
melhor_modelo = grid.best_estimator_
pred = melhor_modelo.predict(X_teste)
print(f'Acurácia no teste: {accuracy_score(y_teste, pred):.2f}')
```



O que documentar no projeto?

Um projeto completo tem que ter:

Seção	O que incluir
1. Introdução	Problema, dataset, objetivo
2. EDA	Visualizações, insights, tratamento de dados
3. Pré-processamento	Etapas aplicadas (nulos, escalonamento, etc.)
4. Modelagem	Modelos testados, código, resultados
5. Comparação	Tabela comparativa, escolha do melhor
6. Conclusão	Resultados finais, aprendizados, próximos passos

O notebook deve ser **autoexplicativo** – alguém que não viu o grupo deve conseguir entender o que foi feito e por quê.

Como organizar a entrega final

```
meu-projeto-ml/  
├── README.md           # Apresentação do projeto  
├── notebooks/  
│   ├── 01_eda_preprocessamento.ipynb  
│   ├── 02_modelagem.ipynb  
│   └── 03_otimizacao.ipynb  
├── dados/  
│   ├── dados_brutos.csv  
│   └── dados_processados.csv  
├── apresentacao/  
│   └── slides_workshop.pdf  
└── minicurso/  
    └── material_apoio.pdf
```

Organização é importante! Facilita a correção e a apresentação.



Resumo da Semana

- Construímos **vários modelos** (Regressão Logística, Árvore, Random Forest, etc.)
- **Comparamos** os modelos com métricas apropriadas
- **Escolhemos o melhor modelo** para o problema
- **Otimizamos** o modelo escolhido com GridSearch
- **Documentamos** todo o processo no notebook
- **Organizamos** o repositório para a entrega final

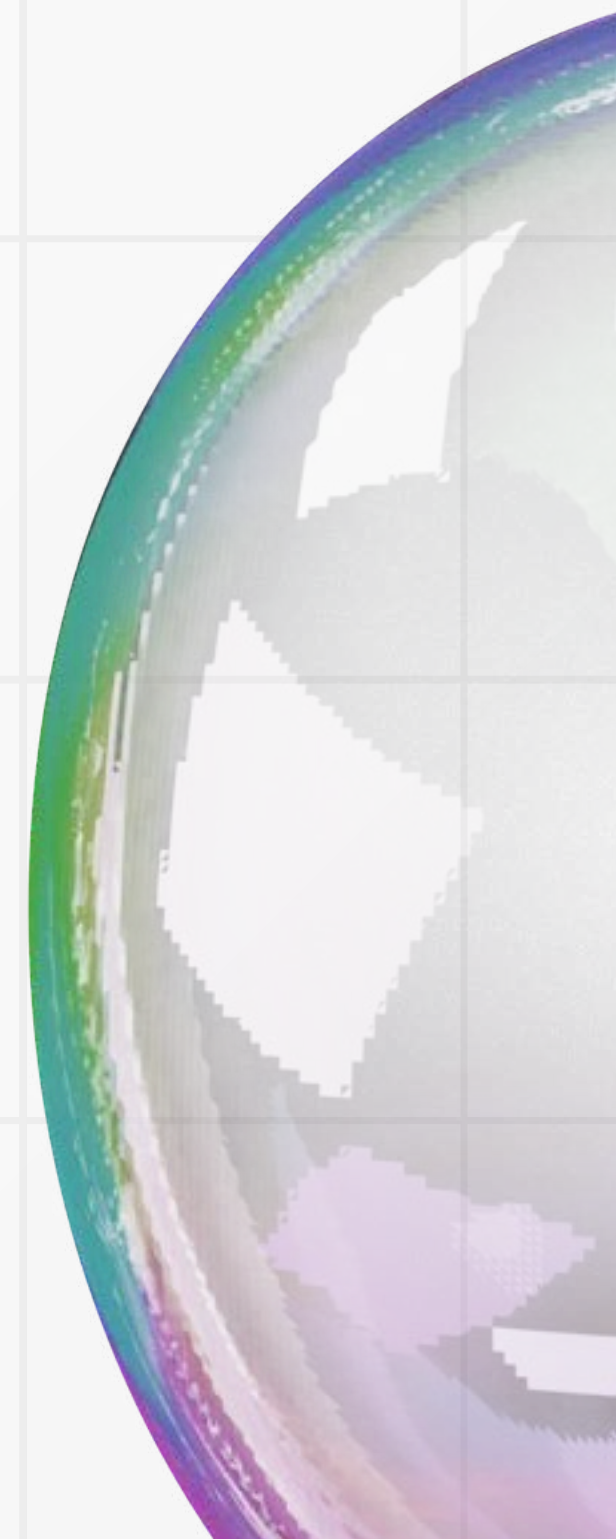


Atividade da Semana

Objetivo: Finalizar o projeto completo.

Instruções:

1. Construir múltiplos modelos no seu dataset (3–5 modelos)
2. Comparar os modelos usando métricas apropriadas
3. Escolher o melhor modelo e justificar a escolha
4. Otimizar o modelo escolhido com GridSearch
5. **Documentar tudo no notebook:**
 - Introdução
 - EDA e pré-processamento (já feito na semana passada)
 - Modelagem e comparação
 - Escolha do melhor modelo
 - Conclusão
6. Organizar o repositório com a estrutura sugerida
7. Subir o projeto completo na pasta da semana 15



Checklist final – Projeto completo

Antes de entregar, verifique:

- Dataset escolhido e variável alvo definida
- EDA com visualizações e insights
- Valores nulos tratados
- Escalonamento aplicado (se necessário)
- Variáveis categóricas codificadas
- Dados divididos em treino e teste
- Baseline treinada
- Pelo menos 3 modelos testados
- Modelo escolhido e justificado
- Otimização com GridSearch aplicada
- Notebook documentado (introdução, seções, comentários)
- Repositório organizado



Dicas para a entrega

- **Não se preocupem com perfeição:** o mais importante é o processo e a documentação. Um modelo com acurácia 0,75 e bem documentado vale mais que 0,95 sem explicação.
- **Se o modelo não for bom, tente entender por quê:** olhe os dados de novo, veja se alguma variável importante foi deixada de lado, teste diferentes abordagens de pré-processamento.
- **Caprichem na documentação:** comentários no código, texto explicativo em markdown, organização do repositório.



Referências

- [Documentação Scikit-learn – Modelos](#)
- [Comparação de modelos – Exemplos](#)
- [Como escrever um bom notebook \(Kaggle\)](#).

